

¡Claro que sí! Aquí tienes el código de tu proyecto con comentarios detallados en cada paquete, clase y método para que te sirva de guía.

He añadido explicaciones sobre el **propósito** de cada elemento y cómo se relaciona con los ejercicios de concurrencia (`@Async`, `CompletableFuture`, etc.).

config/AsyncConfig.java

Este paquete y clase son **fundamentales** para la concurrencia. Aquí es donde defines la "piscina" (Pool) de hilos que ejecutarán tus tareas asíncronas.

Java

```
package com.dam2.Practica1.config;

import org.springframework.context.annotation.*;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;
import java.util.concurrent.Executor;

/**
 * Clase de Configuración de Spring.
 * @Configuration indica a Spring que esta clase contiene definiciones de Beans.
 * @EnableAsync habilita la capacidad de Spring para ejecutar métodos @Async
 * en un pool de hilos en segundo plano. Sin esta anotación, @Async no funciona.
 */
@Configuration
@EnableAsync
public class AsyncConfig {

    /**
     * Define un Bean llamado "taskExecutor".
     * Un Bean es un objeto gestionado por el contenedor de Spring.
     * Este método crea y configura un pool de hilos personalizado.
     *
     * Cuando uses @Async("taskExecutor"), Spring buscará este Bean
     * para ejecutar la tarea.
     *
     * @return El ejecutor de tareas (pool de hilos) configurado.
     */
    @Bean(name = "taskExecutor")
    public Executor taskExecutor() {
        // Creamos un ejecutor de pool de hilos específico de Spring
        ThreadPoolTaskExecutor ex = new ThreadPoolTaskExecutor();

        // Número de hilos a mantener en el pool, incluso si están inactivos.
        // Es el tamaño "núcleo".
        ex.setCorePoolSize(3);
    }
}
```

```

// Número máximo de hilos que el pool puede crear.
// Si la cola se llena, creará nuevos hilos hasta este límite.
ex.setMaxPoolSize(6);

// Capacidad de la cola para tareas pendientes.
// Si llegan más de 6 tareas y las 6 están ocupadas, 20 podrán esperar
// en esta cola antes de que se rechace la tarea.
ex.setQueueCapacity(20);

// Prefijo para los nombres de los hilos.
// ¡Clave para el Ejercicio 1.8! Verás "psp-1", "psp-2" en la consola.
ex.setThreadNamePrefix("psp-");

// Inicializa el ejecutor.
ex.initialize();
return ex;
}
}

```

dto/ImportarPeliculaDto.java

DTO (Data Transfer Object). Un objeto simple para transportar datos. En este caso, se usa para recibir el JSON de la petición POST del Ejercicio 3.

Java

```

package com.dam2.Practica1.dto;

/**
 * Esto es un 'record' de Java. Es una forma moderna y concisa de crear
 * clases DTO (Data Transfer Object) inmutables.
 *
 * Automáticamente genera:
 * - Un constructor que toma todos los campos (String path).
 * - Métodos 'getters' (p.ej., .path()).
 * - Métodos equals(), hashCode() y toString().
 *
 * Su único propósito es recibir el path de la carpeta en la petición POST.
 * Ejemplo de JSON que mapearía a esto:
 * {
 *   "path": "C:/mis-peliculas"
 * }
 */
public record ImportarPeliculaDto(
    String path
) {}

```

models/Pelicula.java

Este es tu modelo de datos (Entidad JPA). Representa la tabla `peliculas` en la base de datos.

Java

```
package com.dam2.Practica1.models;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/**
 * @Entity: Marca esta clase como una entidad JPA (mapeada a una tabla de BD).
 * @Table(name = "peliculas"): Especifica el nombre de la tabla en la BD.
 * @Data: Anotación de Lombok. ¡MUY IMPORTANTE!
 * Genera automáticamente:
 * - Getters para todos los campos.
 * - Setters para todos los campos.
 * - Un método toString().
 * - Métodos equals() y hashCode().
 *
 * NOTA: Los getters y setters manuales que están más abajo son REDUNDANTES
 * debido a que ya usas @Data. Podrías borrarlos.
 */
@Entity
@Table(name = "peliculas")
@Data
public class Pelicula {

    @Id // Marca este campo como la clave primaria (PK).
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Indica que la BD genera el valor
    (autoincremental).
    private Long id;

    @Column(nullable = false, length = 120) // Define propiedades de la columna (no nulo,
    longitud).
    private String titulo;

    private int duracion; // en minutos

    @Column(name = "fecha_estreno") // Mapea este campo a la columna 'fecha_estreno'.
    private LocalDate fechaEstreno;

    private String sinopsis;
```

```

private int valoracion;

// --- Relaciones JPA ---

@OneToOne // Relación 1 a 1: Una Película tiene una FichaTecnica.
@JoinColumn(name = "ficha_id") // Esta tabla (Pelicula) tendrá la clave foránea 'ficha_id'.
private FichaTecnica fichaTecnica;

@ManyToOne // Relación N a 1: Muchas Películas pueden tener un Director.
@JoinColumn(name = "director_id", nullable = false) // La clave foránea 'director_id' está aquí.
private Director director;

@ManyToMany(mappedBy = "peliculas") // Relación N a N: Muchas Películas <-> Muchos
Actores.
// 'mappedBy = "peliculas"' indica que la clase 'Actor' es la dueña
// de la relación (allí estará la definición de la tabla intermedia).
private List<Actor> actors = new ArrayList<>();

// --- Constructores ---

// Constructor con todos los argumentos (Lombok también puede generarlo con
@AllArgsConstructor)
public Pelicula(Long id, String titulo, int duracion, LocalDate fechaEstreno, String sinopsis, int
valoracion, FichaTecnica fichaTecnica, Director director, List<Actor> actors) {
    this.id = id;
    this.titulo = titulo;
    this.duracion = duracion;
    this.fechaEstreno = fechaEstreno;
    this.sinopsis = sinopsis;
    this.valoracion = valoracion;
    this.fichaTecnica = fichaTecnica;
    this.director = director;
    this.actors = actors;
}

// Constructor vacío (sin argumentos).
// Requerido por JPA para crear instancias de la entidad.
// Lombok lo genera con @NoArgsConstructor.
public Pelicula() {
}

// --- Getters y Setters ---
//
// NOTA: Como se mencionó arriba, todos estos métodos (getId, setId, getTitulo, etc.)
// son innecesarios porque la anotación @Data de Lombok ya los genera
// automáticamente en tiempo de compilación.
//
public Long getId() {
    return id;
}

```

```
// ... (resto de getters/setters) ...
public void setActors(List<Actor> actors) {
    this.actors = actors;
}
}
```

controllers/PeliculaController.java

El Controlador. Es la puerta de entrada a tu API. Recibe peticiones HTTP (de un navegador o Postman) y las dirige al `Service` correspondiente.

Java

```
package com.dam2.Practica1.controllers;
```

```
import com.dam2.Practica1.dto.ImportarPeliculaDto;
import com.dam2.Practica1.models.Pelicula;
import com.dam2.Practica1.service.PeliculaService;
import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.concurrent.CompletableFuture;
```

```
/**
 * @RestController: Combina @Controller y @ResponseBody.
 * Indica que esta clase es un controlador REST y que sus métodos
 * devolverán datos (JSON/XML) directamente en el cuerpo de la respuesta.
 *
 * @RequestMapping("/api/peliculas"): Define la URL base para todos
 * los endpoints (métodos) de este controlador.
 *
 * @RequiredArgsConstructor: Anotación de Lombok. Crea un constructor
 * que incluye todos los campos marcados como 'final'.
 * (Aunque aquí estás usando @Autowired en su lugar).
 */
```

```
@RestController
@RequestMapping("/api/peliculas")
@RequiredArgsConstructor
public class PeliculaController {
```

```
// @Autowired: Inyección de Dependencias.
// Spring automáticamente "inyecta" (asigna) una instancia
// del Bean PeliculaService en este campo.
@Autowired
private PeliculaService service;
```

```
// --- Endpoints CRUD básicos ---

// GET /api/peliculas
@GetMapping
public List<Pelicula> listar() {
    return service.listar();
}

// GET /api/peliculas/5 (donde 5 es el id)
@GetMapping("/{id}")
public Pelicula buscarPorId(@PathVariable Long id) {
    return service.buscarPorId(id);
}

// GET /api/peliculas/peliculas-mejores
@GetMapping("/peliculas-mejores")
public List<Pelicula> mejores_peliculas() {
    return service.mejores_peliculas(5);
}

// POST /api/peliculas (con un JSON de Pelicula en el body)
@PostMapping
public void agregar(@RequestBody Pelicula pelicula) {
    service.agregar(pelicula);
}

// --- Endpoints de los Ejercicios de Concurrencia ---

/**
 * Ejercicio 1.2 y 1.3: Endpoint para probar la ejecución SÍNCRONA.
 * GET /api/peliculas/procesar
 */
@GetMapping("/procesar")
public String procesarPeliculas() {
    // Llama al método síncrono del servicio.
    // La petición HTTP se bloqueará hasta que termine (aprox. 9 seg).
    return this.service.procesarPeliculas();
}

/**
 * Ejercicio 1.5: Endpoint para probar la ejecución ASÍNCRONA.
 * GET /api/peliculas/procesar-async
 */
@GetMapping("/procesar-async")
public CompletableFuture<String> procesarAsync() {
    // Llama al método asíncrono. Este método devuelve un CompletableFuture
    // casi al instante. Spring gestiona este 'Future' y enviará
    // la respuesta HTTP solo cuando el 'Future' se complete.
    return this.service.procesarPeliculasAsync();
}

```

```

/**
 * Ejercicio 2: Endpoint para simular reproducción de películas.
 * GET /api/peliculas/reproducir
 */
@GetMapping("/reproducir")
public String reproducirAsync() {
    return this.service.reproducirAsync();
}

/**
 * Ejercicio 3: Endpoint para importar películas desde ficheros.
 * POST /api/peliculas/importar-peliculas
 * Body esperado: { "path": "ruta/a/la/carpeta" }
 */
@PostMapping("/importar-peliculas")
public CompletableFuture<String> importarPeliculas(@RequestBody ImportarPeliculaDto
importarPeliculaDto) {
    try {
        // Llama al servicio de importación, que es asíncrono.
        return this.service.importarPeliculas(importarPeliculaDto.path());
    } catch (Exception e) {
        // Si hay un error (p.ej., la carpeta no existe), devuelve un
        // CompletableFuture "fallido" con la excepción.
        return CompletableFuture.failedFuture(e);
    }
}
}

```

service/PeliculaService.java

La capa de Servicio (Lógica de Negocio). Aquí es donde ocurre la "magia" de los ejercicios. Coordina las tareas, llama al repositorio y maneja la lógica de la aplicación.

Java

```

package com.dam2.Practica1.service;

// ... (imports) ...
import com.dam2.Practica1.models.Pelicula;
import com.dam2.Practica1.repository.PeliculaRepository;
import lombok.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.scheduling.annotation.Async;
import java.util.concurrent.CompletableFuture;
import java.io.*;
import java.nio.file.Files;

```

```

import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.stream.Stream;

/**
 * @Service: Marca esta clase como un Bean de Servicio de Spring.
 * Es donde reside la lógica de negocio.
 * @Getter: Lombok genera getters para los campos.
 */
@Service
@Getter
public class PeliculaService {

    // Inyecta el repositorio JPA para interactuar con la BD.
    @Autowired
    private PeliculaRepository peliculaRepository;

    // ¡MUY IMPORTANTE! Inyecta el OTRA clase de servicio (TareaAsync).
    // Para que @Async funcione, debe ser llamado desde OTRO bean.
    // Llamar a this.metodoAsincrono() NO funciona.
    @Autowired
    private TareaAsync tareaAsync;

    // --- Simulación de BD en memoria ---
    private final List<Pelicula> peliculas = new ArrayList<>();

    public PeliculaService() {
        // Constructor: Rellena la lista en memoria con datos de ejemplo.
        peliculas.add(new Pelicula(1L, "Interstellar", 169, LocalDate.of(2014, 11, 7),
            "Exploradores espaciales buscan un nuevo hogar para la humanidad.", 6, null, null,
null));
        // ... (más películas) ...
    }

    // --- Métodos CRUD (usando la lista en memoria) ---
    public List<Pelicula> mejores_peliculas(int valoracion) {
        // ... (lógica de filtro) ...
        return peliculas_aux;
    }

    public List<Pelicula> listar() {
        return peliculas;
    }

    public Pelicula buscarPorId(Long id) {
        // ... (lógica de búsqueda) ...
        // La versión comentada con Stream es más moderna y funcional.
    }

    public void agregar(Pelicula pelicula) {

```



```

        películas.add(película);
    }

// --- Ejercicio 1: Síncrono vs Asíncrono ---

/**
 * Ejercicio 1.1: Tarea Síncrona.
 * Esta tarea BLOQUEA el hilo que la llama durante 3 segundos.
 */
public String tareaLentaSync(String título) {
    try {
        // Imprime el nombre del hilo actual (p.ej., "http-nio-8080-exec-1")
        System.out.println("Iniciando tarea para " + título + " en " +
            Thread.currentThread().getName());
        Thread.sleep(3000); // BLOQUEO
        System.out.println("Terminando tarea para " + título);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
    return "Procesada " + título;
}

/**
 * Ejercicio 1.2 y 1.3: Orquestador Síncrono.
 * Llama a la tarea lenta 3 veces, una detrás de otra.
 */
public String procesarPelículas() {
    long inicio = System.currentTimeMillis();
    this.tareaLentaSync("Interstellar"); // Espera 3 seg
    this.tareaLentaSync("The Dark Knight"); // Espera 3 seg
    this.tareaLentaSync("Soul"); // Espera 3 seg
    long fin = System.currentTimeMillis();
    // El tiempo total será ~9000 ms (9 segundos).
    return "Tiempo total: " + (fin - inicio) + " ms";
}

/**
 * Ejercicio 1.5: Orquestador Asíncrono.
 * Llama a las tareas asíncronas (definidas en TareaAsync).
 */
public CompletableFuture<String> procesarPelículasAsync() {
    long inicio = System.currentTimeMillis();

    // Llama a las tareas asíncronas. ¡Importante: this.tareaAsync...!
    // Estas llamadas retornan un CompletableFuture INMEDIATAMENTE.
    // Las tareas empiezan a ejecutarse en los hilos "psp-".
    var t1 = this.tareaAsync.tareaLenta2Async("🍿 Interstellar");
    var t2 = this.tareaAsync.tareaLenta2Async("🦇 The Dark Knight");
    var t3 = this.tareaAsync.tareaLenta2Async("🎵 Soul");
    var t4 = this.tareaAsync.tareaLenta2Async("🎵 Soul");

```

```

var t5 = this.tareaAsync.tareaLenta2Async("🎵 Soul");
var t6 = this.tareaAsync.tareaLenta2Async("🎵 Soul");

// Ejercicio 1.6: Espera a que TODAS las tareas terminen.
// .join() bloquea el hilo actual (el del controlador) hasta que
// t1, t2, t3, t4, t5 y t6 se hayan completado.
CompletableFuture.allOf(t1, t2, t3, t4, t5, t6).join();

// Ejercicio 1.7: Calcula el tiempo.
long fin = System.currentTimeMillis();
// El tiempo total será ~6000 ms (6 segundos), porque tienes 6 tareas
// y un pool de 3 hilos (3 se ejecutan, 3 esperan -> 3s + 3s = 6s).
// Si aumentas el CorePoolSize a 6, el tiempo total sería ~3000 ms.
return CompletableFuture.completedFuture("Tiempo total (asíncrono): " + (fin - inicio) + "
ms");
}

// --- Ejercicio 2: Reproducción de Películas ---

public String reproducirAsync() {
    long inicio = System.currentTimeMillis();

    // ¡¡OJO!! ¡ESTO ES UNA AUTO-LLAMADA (self-invocation)!
    // Estás llamando a this.reproducir().
    // Como 'reproducir' está en la *misma clase*, la anotación @Async
    // será IGNORADA por Spring.
    //
    // *** ESTA EJECUCIÓN SERÁ SÍNCRONA ***
    //
    // Para que 'reproducir' fuera asíncrono, deberías moverlo
    // a la clase TareaAsync, igual que hiciste con tareaLenta2Async.
    var t1 = this.reproducir("🎬 Interstellar");
    var t2 = this.reproducir("🦇 The Dark Knight");
    var t3 = this.reproducir("🎵 Soul");

    // Como las llamadas fueron síncronas, cuando el código llega aquí,
    // todas las tareas (t1, t2, t3) YA HAN TERMINADO.
    // El .join() no espera nada porque los futures ya están completados.
    CompletableFuture.allOf(t1, t2, t3).join();

    long fin = System.currentTimeMillis();
    return "Tiempo total (asíncrono): " + (fin - inicio) + " ms";
}

/**
 * Ejercicio 2: Método de reproducción (con el problema de auto-invocación).
 *
 * @Async("taskExecutor") -> Esta anotación NO FUNCIONARÁ
 * si se llama desde un método de esta misma clase (como reproducirAsync).
 */

```

```

@Async("taskExecutor")
public CompletableFuture<String> reproducir(String titulo) {
    Long inicio = System.currentTimeMillis();
    try {
        System.out.println("Reproduciendo " + titulo + " en " +
Thread.currentThread().getName());
        // Genera un tiempo aleatorio entre 1 y 5 segundos (1000ms a 5000ms)
        int timeX = (int) (Math.random() * 4001) + 1000; // Corrección: (max - min + 1) + min ->
(5000-1000+1) + 1000
        Thread.sleep(timeX);
        System.out.println("Terminando " + titulo);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
    Long fin = System.currentTimeMillis();
    return CompletableFuture.completedFuture("Procesada " + titulo + " en " + ((fin - inicio) /
1000) + " segundos");
}

```

// --- Ejercicio 3: Importar Ficheros ---

```

/**
 * Orquestador de importación de ficheros.
 * Lee un directorio y lanza una tarea asíncrona por cada fichero.
 */
public CompletableFuture<String> importarPeliculas(String rutaCarpeta) throws IOException {
    long inicio = System.currentTimeMillis();
    List<CompletableFuture<Void>> futures = new ArrayList<>();

    // Usa try-with-resources para listar los ficheros de la carpeta.
    // Esto asegura que el Stream se cierre correctamente.
    try (Stream<Path> paths = Files.list(Paths.get(rutaCarpeta))) {
        paths
            .filter(Files::isRegularFile) // Asegura que solo procesa ficheros (no carpetas)
            .forEach(path -> {
                String nombre = path.toString().toLowerCase();
                if (nombre.endsWith(".csv") || nombre.endsWith(".txt")) {
                    // Lanza la tarea ASÍNCRONA de importar CSV
                    // (correctamente, llamando a tareaAsync)
                    futures.add(this.tareaAsync.importarCsvAsync(path));
                } else if (nombre.endsWith(".xml")) {
                    // Lanza la tarea ASÍNCRONA de importar XML
                    futures.add(this.tareaAsync.importarXmlAsync(path));
                }
            });
    }
}

```

```

// Espera a que TODAS las tareas de importación (CSV y XML) terminen.
CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();

```

```

    long fin = System.currentTimeMillis();
    // El tiempo total será el del fichero que más tarde en procesarse,
    // suponiendo que tengas suficientes hilos en el pool.
    return CompletableFuture.completedFuture("Importación completa en " + (fin - inicio) + "
ms");
}
}

```

service/TareaAsync.java

Clase de servicio auxiliar. Su **único propósito** es contener los métodos `@Async` para que puedan ser llamados correctamente desde `PeliculaService` y así activar la concurrencia.

Java

```

package com.dam2.Practica1.service;

import com.dam2.Practica1.models.Pelicula;
import com.dam2.Practica1.repository.PeliculaRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.NodeList;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CompletableFuture;

/**
 * @Service
 * Esta clase existe para albergar los métodos @Async y evitar
 * el problema de la "auto-invocación" (self-invocation).
 * PeliculaService llama a los métodos de esta clase para
 * ejecutar tareas en hilos separados.
 */
@Service
public class TareaAsync {

    @Autowired
    private PeliculaRepository peliculaRepository;

```

```

/**
 * Ejercicio 3: Tarea asíncrona para procesar un fichero CSV/TXT.
 *
 * @Async("taskExecutor") le dice a Spring: "Ejecuta este método
 * en un hilo del pool 'taskExecutor' que definimos en AsyncConfig".
 * @return un CompletableFuture<Void> que se completará cuando la
 * importación termine.
 */
@Async("taskExecutor")
public CompletableFuture<Void> importarCsvAsync(Path fichero) {
    try {
        // Imprime el nombre del hilo (p.ej., "psp-1")
        System.out.println("Procesando CSV: " + fichero + " en " +
            Thread.currentThread().getName());

        List<Película> lista = new ArrayList<>();
        List<String> lineas = Files.readAllLines(fichero); // Lee todas las líneas
        lineas.remove(0); // Asume que la primera línea es un encabezado y la descarta.

        // Itera sobre las líneas y "parsea" (convierte) el texto a objetos Película.
        for (String linea : lineas) {
            String[] campos = linea.split(";"); // Separa por punto y coma
            Película p = new Película();
            p.setTitulo(campos[0]);
            p.setDuracion(Integer.parseInt(campos[1]));
            p.setFechaEstreno(LocalDate.parse(campos[2]));
            p.setSinopsis(campos[3]);
            lista.add(p);
        }

        // Guarda todas las películas parseadas en la base de datos
        // (Esta podría ser una operación lenta, por eso es asíncrona).
        películaRepository.saveAll(lista);

        System.out.println("Finalizado CSV: " + fichero);

    } catch (Exception e) {
        System.err.println("Error en CSV " + fichero + ": " + e.getMessage());
    }

    // Devuelve un Future completado para indicar que la tarea ha finalizado.
    return CompletableFuture.completedFuture(null);
}

/**
 * Ejercicio 3: Tarea asíncrona para procesar un fichero XML.
 * Funciona igual que la de CSV, pero usa un parser DOM para XML.
 */
@Async("taskExecutor")

```

```

public CompletableFuture<Void> importarXmlAsync(Path fichero) {
    try {
        System.out.println("Procesando XML: " + fichero + " en " +
            Thread.currentThread().getName());

        List<Película> lista = new ArrayList<>();

        // Configuración estándar del parser DOM de Java.
        DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
        DocumentBuilder builder = factory.newDocumentBuilder();

        // Parsea el fichero XML
        Document doc = builder.parse(fichero.toFile());
        // Obtiene una lista de todos los nodos <película>
        NodeList nodos = doc.getElementsByTagName("película");

        // Itera sobre cada nodo <película>
        for (int i = 0; i < nodos.getLength(); i++) {
            Element e = (Element) nodos.item(i); // Obtiene el elemento

            // Extrae el texto de las etiquetas hijas (título, duración, etc.)
            Película p = new Película();
            p.setTítulo(e.getElementsByTagName("título").item(0).getTextContent());

            p.setDuración(Integer.parseInt(e.getElementsByTagName("duración").item(0).getTextContent()));

            p.setFechaEstreno(LocalDate.parse(e.getElementsByTagName("fechaEstreno").item(0).getTextContent()));
            p.setSinopsis(e.getElementsByTagName("sinopsis").item(0).getTextContent());

            lista.add(p);
        }

        // Guarda en la BD
        películaRepository.saveAll(lista);

        System.out.println("Finalizado XML: " + fichero);

    } catch (Exception e) {
        System.err.println("Error en XML " + fichero + ": " + e.getMessage());
    }

    return CompletableFuture.completedFuture(null);
}

/**
 * Ejercicio 1.4: Tarea Asíncrona Lenta.
 * Esta es la versión asíncrona de 'tareaLentaSync'.
 * Será ejecutada en un hilo del pool "psp-".
 */

```

```

@Async("taskExecutor")
public CompletableFuture<String> tareaLenta2Async(String titulo) {
    try {
        // Imprime el nombre del hilo (p.ej., "psp-2")
        System.out.println("Iniciando " + titulo + " en " + Thread.currentThread().getName());
        // Duerme el hilo "psp-2", pero NO el hilo principal.
        Thread.sleep(3000);
        System.out.println("Terminando " + titulo);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
    // Devuelve un Future completado con el resultado.
    return CompletableFuture.completedFuture("Procesada " + titulo);
}
}

```

Espero que esta guía detallada te sea muy útil para tus próximos proyectos. He destacado los puntos clave de la concurrencia, como la configuración del `ThreadPool`, el uso de `CompletableFuture`, y el error común de la auto-invocación de `@Async`.

¿Te gustaría que profundicemos en algún punto específico, como la diferencia entre `CompletableFuture.allOf` y `CompletableFuture.anyOf`?